

Linux Shell scriptek – Összefoglaló

A **Linux Shell script** egy olyan szöveges fájl, amely parancsokat tartalmaz egy Unix-alapú operációs rendszer **parancsértelmezőjének (Shell)**. Ezek a scriptek lehetővé teszik az automatizálást, az ismétlődő feladatok egyszerűsítését és a rendszerfelügyelet hatékonyabbá tételét.

1. Mi az a Shell?

A **Shell** egy parancsértelmező, amely lehetővé teszi a felhasználó számára, hogy interakcióba lépjen az operációs rendszerrel. Többféle Shell létezik, például:

- **Bash (Bourne Again Shell)** – A legnépszerűbb és leggyakrabban használt Shell a Linux rendszerekben.
 - **Sh (Bourne Shell)** – Az eredeti Unix Shell.
 - **Zsh (Z Shell)** – Fejlettebb, testreszabhatóbb Shell.
 - **Ksh (Korn Shell)** – Teljesítményorientált és scriptelésre optimalizált Shell.
-

2. Shell scriptek alapjai

Egy **Shell script** egy egyszerű szövegfájl, amely parancsokat tartalmaz. A fájl neve általában `.sh` végződésű.

Példa egy egyszerű Bash scriptre:

```
#!/bin/bash
echo "Hello, világ!"
```

Magyarázat:

- `#!/bin/bash` – Shebang: Megadja, hogy a script a Bash-t használja.
- `echo "Hello, világ!"` – Kiírja a "Hello, világ!" szöveget a terminálra.

A script futtathatóvá tétele:

```
chmod +x script.sh
./script.sh
```

3. Változók és alapvető parancsok

Változók deklarálása:

```
név="Linux"
echo "Üdvözlünk, $név!"
```

Felhasználói bemenet fogadása:

```
echo "Mi a neved?"
read nev
echo "Szia, $nev!"
```

Elágazások (if-else):

```
#!/bin/bash
echo "Add meg a jelszót:"
read -s jelszo

if [ "$jelszo" == "titok" ]; then
    echo "Hozzáférés engedélyezve!"
else
    echo "Hibás jelszó!"
fi
```

Ciklusok:

for ciklus:

```
for i in {1..5}; do
    echo "Ez a $i. sor"
done
```

while ciklus:

```
szam=1
while [ $szam -le 5 ]; do
    echo "Szám: $szam"
    ((szam++))
done
```

4. Függvények a Shell scriptekben

A függvények segítenek a kód modularizálásában és újrafelhasználásában.

```
#!/bin/bash
üdvözlés() {
    echo "Szia, $1!"
}

üdvözlés "Gábor"
```

5. Shell scriptek felhasználási területei

Rendszeradminisztráció – Naplófájlok kezelése, automatikus mentések, szerverkonfigurációk.

Automatizálás – Ismétlődő feladatok, például szoftvertelepítés, adatfeldolgozás.

Adatkezelés – Nagyméretű szöveges fájlok vagy adatbázisok elemzése.

Hálózati műveletek – Ping tesztek, hálózati forgalom monitorozása, szerverkapcsolatok ellenőrzése.

6. Néhány egyszerű de hasznos Shell script:

6.1. Fájltypusok számolása egy könyvtárban

Ez a script megszámolja, hogy egy adott könyvtárban hány darab adott kiterjesztésű fájl található.

Script:

```
#!/bin/bash
echo "Add meg a könyvtár elérési útját:"
read konyvtar
echo "Add meg a fájlkiterjesztést (pl. txt, sh, log):"
read kiterjesztes

szam=$(ls "$konyvtar"/*."$kiterjesztes" 2>/dev/null | wc -l)
echo "A(z) $konyvtar könyvtárban $szam darab .$kiterjesztes fájl található."
```

Magyarázat:

- `read konyvtar` – Bekéri a felhasználótól a könyvtár nevét.
- `read kiterjesztes` – Bekéri a kívánt fájltypust.
- `ls "$konyvtar"/*."$kiterjesztes"` – Kijelentkeztet az adott típusú fájlokat.
- `wc -l` – Megszámolja a fájlokat.
- `2>/dev/null` – Az esetleges hibaüzeneteket elnyeli (ha nincs ilyen fájl).

2. Automatikus biztonsági mentés

Ez a script biztonsági mentést készít egy adott könyvtárról egy másik helyre egy időbélyeggel ellátott ZIP fájlba.

Script:

```
#!/bin/bash
mentesi_hely="/home/$USER/backup"
forras_konyvtar="/home/$USER/Documents"

# Ellenőrizzük, hogy létezik-e a mentési könyvtár
if [ ! -d "$mentesi_hely" ]; then
    mkdir -p "$mentesi_hely"
fi

# Fájlnev az aktuális dátummal
datum=$(date +%Y-%m-%d_%H-%M-%S)
tar -czf "$mentesi_hely/backup_$datum.tar.gz" "$forras_konyvtar"

echo "A biztonsági mentés elkészült: $mentesi_hely/backup_$datum.tar.gz"
```

Magyarázat:

- `mentesi_hely="/home/$USER/backup"` – A mentés célkönyvtára.
- `forras_konyvtar="/home/$USER/Documents"` – A menteni kívánt könyvtár.
- `mkdir -p "$mentesi_hely"` – Ha a mentési könyvtár nem létezik, létrehozza.
- `date +%Y-%m-%d_%H-%M-%S` – Az aktuális dátum és idő a fájlnevhez.

- `tar -czf` – A kiválasztott könyvtárat tömöríti egy időbélyeges .tar.gz fájlba.

3. Nagyméretű fájlok keresése

Ez a script megkeresi az 1 GB-nál nagyobb fájlokat egy adott könyvtárban.

Script:

```
#!/bin/bash
echo "Add meg a vizsgálandó könyvtár elérési útját:"
read konyvtar

echo "1 GB-nál nagyobb fájlok listázása a(z) $konyvtar könyvtárban:"
find "$konyvtar" -type f -size +1G
```

Magyarázat:

- `find "$konyvtar" -type f -size +1G` – Megkeresi az 1 GB-nál nagyobb fájlokat az adott könyvtárban.
- `-type f` – Csak fájlokat keres, könyvtárakat nem.
- `-size +1G` – Csak az 1 GB-nál nagyobb fájlokat listázza.

4. Hálózati kapcsolat ellenőrzése (ping teszt)

Ez a script megvizsgálja, hogy egy adott szerver (például google.com) elérhető-e.

Script:

```
#!/bin/bash
echo "Add meg a weboldalt vagy IP címet, amit tesztelni szeretnél:"
read cim

ping -c 4 "$cim" > /dev/null

if [ $? -eq 0 ]; then
    echo "A(z) $cim elérhető."
else
    echo "A(z) $cim nem elérhető."
fi
```

Magyarázat:

- `ping -c 4 "$cim"` – 4 csomagot küld a megadott címre.
- `> /dev/null` – A kimenetet elnyeli, hogy ne zavarja a scriptet.
- `?` – Az utolsó parancs végrehajtásának státusza (0 = siker, 1 = sikertelen).

5. Futtató jogok beállítása több fájlra

Ez a script megkeresi az adott könyvtárban lévő .sh fájlokat, és végrehajthatóvá teszi őket.

Script:

```
#!/bin/bash
echo "Add meg a könyvtár elérési útját:"
read konyvtar

chmod +x "$konyvtar"/*.sh
echo "A(z) $konyvtar könyvtárban található összes Shell script futtathatóvá vált."
```

Magyarázat:

- `chmod +x "$konyvtar"/*.sh` – Az adott könyvtárban lévő összes .sh fájlt végrehajthatóvá teszi.
- Ez akkor hasznos, ha több scriptet is le szeretnénk futtatni egy szerveren vagy munkaállomáson.

6. Rendszerinformációk Gyors Ellenőrzése

Ez a script kiírja az alapvető rendszerinformációkat.

Script:

```
#!/bin/bash
echo "Rendszerinformációk:"
echo "-----"
echo "Dátum: $(date)"
```

```
echo "Felhasználó: $USER"
echo "Operációs rendszer: $(uname -o)"
echo "Kernel verzió: $(uname -r)"
echo "Uptime: $(uptime -p)"
echo "Szabad lemezterület:"
df -h | grep "^/dev"
```

Magyarázat:

- `date` – Az aktuális dátum és idő kiírása.
- `$USER` – A jelenlegi felhasználó.
- `uname -o` – Az operációs rendszer típusa.
- `uname -r` – A kernel verziója.
- `uptime -p` – Mennyi ideje fut a rendszer.
- `df -h | grep "^/dev"` – Kiírja a csatlakoztatott meghajtók szabad területét.

Feladatok ütemezése: a cron használata

A cron egy időzített feladatütemező, amely lehetővé teszi a parancsok vagy scriptek bizonyos időközönkénti végrehajtását.

Lépések egy biztonsági mentés script (`backup.sh`) automatizálásához:

1.2. A cron konfigurálása

A felhasználói cron feladatokat az alábbi paranccsal lehet szerkeszteni:

```
crontab -e
```

Ez megnyitja az aktuális felhasználóhoz tartozó **crontab** fájlt egy szövegszerkesztőben.

1.3. Feladat beállítása napi futtatásra

A crontab fájlban adjunk hozzá egy új sort:

```
0 2 * * * /home/user/backup.sh >> /var/log/backup.log 2>&1
```

Ez azt jelenti, hogy a `/home/user/backup.sh` minden nap **hajnali 2:00-kor** lefut, és a kimenetet a `/var/log/backup.log` fájlba menti.

Magyarázat a cron időformátumára:

```
0 2 * * * /home/user/backup.sh
```

- `0 2 * * *` → Minden nap 2:00-kor futtatja a scriptet.

1.4. A beállított cron feladatok ellenőrzése

Ha meg szeretnénk nézni, milyen időzített feladatok vannak beállítva:

```
crontab -l
```

1.5. Naplózás és hibakeresés

Ha a script nem fut megfelelően, érdemes megnézni a naplófájlt:

```
cat /var/log/backup.log
```

Ha a script nem fut le, ellenőrizzük a **syslog** fájlt is:

```
grep CRON /var/log/syslog
```

Ez segíthet az ütemezési problémák felderítésében.

Néhány alapvető példa script:

```
#!/bin/bash
# 1. Egyszerű "Hello, World!" script
# Ez a script egyszerűen kiír egy üzenetet a konzolra.
echo "Hello, World!"

# 2. Változók használata
# A script egy változót hoz létre és kiírja annak értékét.
name="Felhasználó"
echo "Hello, $name!"

# 3. Felhasználói bemenet kezelése
# A script bekéri a felhasználó nevét és kiírja azt.
echo "Mi a neved?"
read username
echo "Üdv, $username!"

# 4. If-else szerkezet használata
# A script ellenőrzi, hogy egy fájl létezik-e.
echo "Adj meg egy fájlnevet:"
read filename
if [ -f "$filename" ]; then
    echo "A fájl létezik."
else
    echo "A fájl nem található."
fi

# 5. For ciklus
# A script végigmegy egy lista elemein és kiírja azokat.
for i in {1..5}; do
    echo "Szám: $i"
done

# 6. While ciklus
# A script 5 másodpercen keresztül másodpercenként kiír egy üzenetet.
counter=1
while [ $counter -le 5 ]; do
    echo "Másodperc: $counter"
    sleep 1
    ((counter++))
done

# 7. Függvények használata
# A script egy függvényt definiál, amely üdvözlí a felhasználót.
hello() {
    echo "Szia, $1!"
}
hello "Béla"

# 8. Parancssori argumentumok kezelése
# A script kiírja az első argumentumot, amelyet meghíváskor adtunk meg.
echo "Az első argumentum: $1"

# 9. Fájlok tartalmának keresése
# A script ellenőrzi, hogy egy fájl tartalmazza-e a "hiba" szót.
echo "Adj meg egy fájlnevet:"
read file_to_search
grep -q "hiba" "$file_to_search" && echo "A fájl tartalmazza a 'hiba' szót." || echo "A fájl nem tartalmazza a 'hiba' szót."
```

```
# 10. Egyszerű menü készítése
# A script egy interaktív menüt biztosít a felhasználónak.
echo "Válassz egy opciót:"
echo "1) Dátum megjelenítése"
echo "2) Aktuális könyvtár"
echo "3) Kilépés"
read choice
case $choice in
    1) date ;;
    2) pwd ;;
    3) echo "Kilépés..."; exit ;;
    *) echo "Érvénytelen választás." ;;
esac
```